



BITS Pilani

Computational Learning Theory

Pooja Harde

What is PAC learning?



- ▶ The material in this lecture is based largely on Chapter 2 from the book Foundations of Machine Learning by Mehryar Mohri, Afhsin Rostamizadeh and Ameet Talwalkar
- ▶ Some of the questions we are interested in the course are the following:
 - ▶ what can be learned efficiently?
 - ▶ what is inherently hard to learn?
 - ▶ how many examples are needed to learn successfully?
 - ▶ Is there a general model of learning?



- ▶ Let $\mathcal{X}$ denote the set of all possible examples or instances. We sometimes call this the input space.
- ▶ The set of all possible labels is denoted by $\mathcal{Y}$. We shall limit ourselves to $\mathcal{Y} = \{0, 1\}$, which corresponds to binary classification.
- ▶ A concept $c : \mathcal{X} \rightarrow \mathcal{Y}$ is a mapping from $\mathcal{X}$ to $\mathcal{Y}$.
- ▶ An example of a concept could be the indicator function of all points in the plane that lie within a triangle. We say that the concept to learn is a triangle.
- ▶ A concept class $\mathcal{C}$ is a set of concepts we wish to learn – a concept class could be the set of all triangles in the plane.



- ▶ We assume that the examples are independent and identically distributed. They are drawn from some distribution $\mathcal{D}$ which is unknown.
- ▶ The learner assumes that the hypothesis set $\mathcal{H}$ consists of a fixed set of concepts, which might not necessarily coincide with $\mathcal{C}$.
- ▶ It receives a sample $S = (x_1, x_2, \dots, x_m)$ and labels associated with this sample $(c(x_1), c(x_2), \dots, c(x_m))$, where c is the specific concept that needs to be learned.
- ▶ The task is to use the sample S to learn a hypothesis h_S such that this hypothesis has a small generalization error, with respect to the actual concept c .



- ▶ Given a hypothesis $h \in \mathcal{H}$, a target concept $c \in \mathcal{C}$ and an underlying distribution $\mathcal{D}$, the generalization error or risk of h is defined as follows:

$$R(h) = \mathbb{P}_{x \sim \mathcal{D}}[h(x) \neq c(x)] = E_{x \sim \mathcal{D}}[1_{h(x) \neq c(x)}]$$

- ▶ The generalization error $R(h)$ of a hypothesis is not known since both the distribution $\mathcal{D}$ and the true concept c are unknown.
- ▶ However the learner can calculate the empirical error of a hypothesis on the labeled sample S



- ▶ Given a hypothesis $h \in \mathcal{H}$, a target concept $c \in \mathcal{C}$ and a sample $S = (x_1, x_2, \dots, x_m)$, the empirical risk of h is defined by

$$\hat{R}_S(h) = \frac{1}{m} \sum_{i=1}^{i=m} 1[h(x_i) \neq c(x_i)]$$

- ▶ The empirical error of $h \in \mathcal{H}$ is the average error of h over the sample S while the generalization error is the expected error of the hypothesis over the distribution $\mathcal{D}$.
- ▶ We would like to explore the relationship between the empirical error and the generalization error.

Generalization error and empirical error



- ▶ We can show the following relationship: $E_{S \sim \mathcal{D}^m}[\hat{R}_S(h)] = R(h)$.
- ▶ To show the above result, we can write:

$$\begin{aligned} E_{S \sim \mathcal{D}^m}[\hat{R}_S(h)] &= E_{S \sim \mathcal{D}^m}\left(\frac{1}{m} \sum_{i=1}^m 1[h(x_i) \neq c(x_i)]\right) \\ &= \frac{1}{m} \sum_{i=1}^m E_{S \sim \mathcal{D}^m} 1[h(x_i) \neq c(x_i)] \\ &= \frac{1}{m} \sum_{i=1}^m E_{S \sim \mathcal{D}^m} 1[h(x) \neq c(x)] \\ &= E_{S \sim \mathcal{D}^m} 1[h(x) \neq c(x)] \end{aligned}$$

PAC learning definition



- ▶ Let n be a number such that any $x \in \mathcal{X}$ can be represented in a computational cost of $O(n)$. For example if x is a vector in $\mathbb{R}^n$, the cost of an array-based representation would be $O(n)$.
- ▶ Let $\text{size}(c)$ represent the computational cost of representing the concept $c \in \mathcal{C}$.
- ▶ Let h_S be the hypothesis returned by an algorithm $\mathcal{A}$ operating on a labeled sample S . We do not explicitly show the dependency of h_S on the algorithm $\mathcal{A}$.
- ▶ With the above foundation, we are now in a position to define the PAC (Probably Approximately Correct) Learning model.



- ▶ A concept class $\mathcal{C}$ is said to be PAC-learnable if there exists an algorithm $\mathcal{A}$ and a polynomial $\text{poly}(\dots)$ such that for any $\epsilon > 0$ and $\delta > 0$, for all distributions $\mathcal{D}$ on $\mathcal{X}$ and for any target concept $c \in \mathcal{C}$, the following holds for any sample size $m \geq \text{poly}(1/\epsilon, 1/\delta, n, \text{size}(c))$:

$$\mathbb{P}_{S \sim \mathcal{D}^m}[R(h_S) < \epsilon] \geq 1 - \delta$$

- ▶ If further $\mathcal{A}$ runs in $\text{poly}(1/\epsilon, 1/\delta, n, \text{size}(c))$, then $\mathcal{C}$ is said to be efficiently PAC-learnable. When such an algorithm $\mathcal{A}$ exists, it is called a PAC-learning algorithm for $\mathcal{C}$.

PAC learning framework



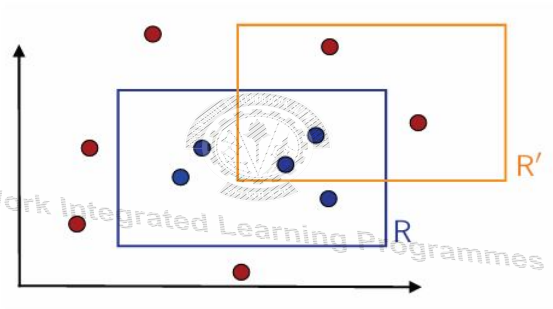
- ▶ The parameter $\delta > 0$ is used to determine the confidence $1 - \delta$ and $\epsilon > 0$, the accuracy $1 - \epsilon$.
- ▶ The PAC learning framework is a distribution-free model, in that no assumption is made regarding the distribution $\mathcal{D}$ from which the samples are drawn.
- ▶ The training and test examples are drawn from the same distribution $\mathcal{D}$. This is an assumption we need to make for generalization to be possible.
- ▶ The PAC framework deals with learnability for a particular concept class $\mathcal{C}$ and not a particular concept.

Example - learning axis aligned rectangles



- ▶ Consider the case where the set of instances are points on the 2D plane, ie $\mathcal{X} = \mathbb{R}^2$.
- ▶ Let the concept class $\mathcal{C}$ be the set of all axis-aligned rectangles lying in $\mathbb{R}^2$.
- ▶ Each concept c is the set of points lie within the corresponding rectangle.
- ▶ The learning problem consists of determining with a small error the target axis-aligned rectangle using the labeled training sample.

Example - learning axis aligned rectangles



Example - learning axis aligned rectangles



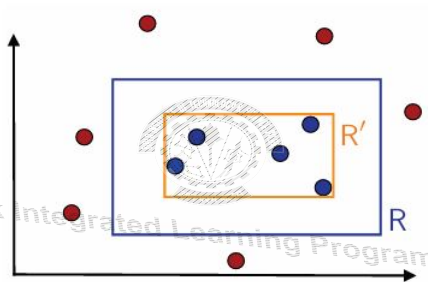
- ▶ In the figure on the previous slide, R represents the true concept and R' represents a possible hypothesis.
- ▶ Circles represent training instances.
- ▶ Points falling within R are blue and labeled 1, while those outside R are red and labeled 0.
- ▶ Points within R' but outside R are false positives as the possible hypothesis is labeling them to be true while they are actually false.
- ▶ Points within R but outside R' are false negatives as the possible hypothesis is labeling them to be false while they are actually true.

Example - learning axis aligned rectangles



- ▶ We shall now show that this concept class is PAC-learnable.
- ▶ The following is a simple PAC-learning algorithm $\mathcal{A}$ - given a labeled sample S , the algorithm consists of returning the tightest axis-aligned rectangle $R' = R_S$ containing the points labeled 1.
- ▶ Given this definition of R' we see that it does not contain any false positives, since any points it labels as 1 will also be labeled by R as 1, as $R' \subset R$.
- ▶ The hypothesis returned by $\mathcal{A}$ is shown in the figure on the next slide.

Example - learning axis aligned rectangles



Example - learning axis aligned rectangles

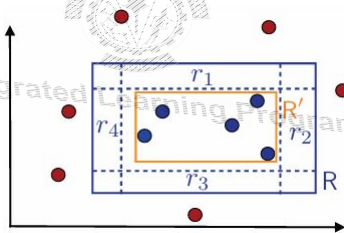


- ▶ Let $R \in \mathcal{C}$ be a target concept.
- ▶ Fix $\epsilon > 0$. Let $\mathbb{P}[R]$ denote the probability that a point drawn from the distribution $\mathcal{D}$ belongs to R , i.e it falls within the region denoted by R .
- ▶ Our algorithm chooses R' to be a subset of R . This means that we cannot have any red points (those labeled 0) in R' as those points would belong to R as well and R can only enclose blue points as it is the true concept.
- ▶ If R' makes an error, it is because it does not enclose some blue points which fall in R .
- ▶ We can assume $\mathbb{P}[R] > \epsilon$, as otherwise $\mathbb{P}[R'] < \mathbb{P}[R] < \epsilon$ and the error of R' is less than ϵ regardless of the sample S received.

Example - learning axis aligned rectangles



- Since $\mathbb{P}[R] > \epsilon$, we can define four regions r_1, r_2, r_3, r_4 along the sides of R such that the probability mass in each of them is at least $\frac{\epsilon}{4}$. To construct each of these regions we can start from a side of R and move the side far enough to enclose a probability mass of at least $\frac{\epsilon}{4}$ as in the figure below:



Example - learning axis aligned rectangles



- ▶ Let l, b, r, t be four values that define the axis-parallel rectangle R , so that $R = [l, r] \times [b, t]$.
- ▶ The left rectangle r_4 is defined by $[l, s_4] \times [b, t]$ where $s_4 = \inf \{s : |\mathbb{P}[l, s] \times [b, t]| \geq \frac{\epsilon}{4}\}$.
- ▶ Let $\bar{r}_4$ be the region obtained by omitting the right boundary of r_4 , i.e $\bar{r}_4 = [l, s_4[\times [b, t]$.
- ▶ Clearly the probability of the region $\bar{r}_4$ is greater than or equal to $\frac{\epsilon}{4}$.
- ▶ We define the regions r_1, r_2, r_3 and the corresponding regions $\bar{r}_1, \bar{r}_2, \bar{r}_3$ similarly.

Example - learning axis aligned rectangles



- ▶ Suppose $R_S = R'$, the hypothesis produced by algorithm $\mathcal{A}$, meets all the four regions $r_i, i \in [4]$. Then, since it is a rectangle, one side of R_S must be in each of these regions.
- ▶ The error area of R_S which is the part of R that it does not cover is included in the union of the regions $\bar{r}_i, i \in [4]$.
- ▶ The probability mass of the union of the regions $\bar{r}_i, i \in [4]$ cannot exceed ϵ .
- ▶ Thus if we take statement A to mean " R_S has a boundary in each of the four regions $r_i, i \in [4]$ ", and statement B to mean "the union of the regions $\bar{r}_i, i \in [4]$ has probability mass less than ϵ ", then what we have is $A \implies B$.
- ▶ Note that B can be rewritten as " $R(R_S) < \epsilon$ "

Example - learning axis aligned rectangles



- ▶ Contraposition of the statement $A \implies B$ gives $\neg B \implies \neg A$.
- ▶ Now $\neg B$ means " $R(R_S) \geq \epsilon$ and $\neg A$ means " R_S must miss at least one of the regions $r_i, i \in [4]$ ".
- ▶ Thus we have:

$$\begin{aligned}\mathbb{P}_{S \sim \mathcal{D}^m}[R(R_S) > \epsilon] &\leq \mathbb{P}_{S \sim \mathcal{D}^m}[\bigcup_{i=1}^{i=4} \{R_S \cap r_i = \phi\}] \\ &\leq \sum_{i=1}^{i=4} \mathbb{P}_{S \sim \mathcal{D}^m}[\{R_S \cap r_i = \phi\}] \\ &\leq 4\left(1 - \frac{\epsilon}{4}\right)^m \\ &\leq 4e^{-\frac{m\epsilon}{4}}\end{aligned}$$

Example - learning axis aligned rectangles



- ▶ The second inequality on the previous slide is a statement of the union bound, i.e. $P(A \cup B \cup C) \leq P(A) + P(B) + P(C)$.
- ▶ The third inequality is true since $P(r_i) \geq \frac{\epsilon}{4}$. The only way for $\{R_S \cap r_i = \phi\}$ to be true, or R_S not meeting r_i is for R_S to be a subset of the complement of r_i , i.e. r_i^C . Since $P(r_i) \geq \frac{\epsilon}{4}$, we must have $\mathbb{P}[\{R_S \cap r_i = \phi\}] = \mathbb{P}[R_S \subset r_i^C] \leq 1 - \frac{\epsilon}{4}$.
- ▶ For the last step we use the inequality that $1 - x \leq e^{-x}$.
- ▶ For any $\delta > 0$, in order to ensure $\mathbb{P}_{S \sim \mathcal{D}^m}[R(R_S) > \epsilon] \leq \delta$ we can set $4e^{-\frac{\epsilon m}{4}} \leq \delta$ which can be rewritten as $m \geq \frac{4}{\epsilon} \log \frac{4}{\delta}$.
- ▶ For any $\epsilon > 0, \delta > 0$, if the sample size $m > \frac{4}{\epsilon} \log \frac{4}{\delta}$, then $\mathbb{P}_{S \sim \mathcal{D}^m}[R(R_S) > \epsilon] \leq \delta$.

PAC-learnability of axis aligned rectangles



- ▶ The computational cost of representing points in $\mathbb{R}^2$ and axis-aligned rectangles (which can be defined by their four corners) is constant.
- ▶ This proves that the concept class of axis-aligned rectangles is PAC-learnable.
- ▶ The sample complexity of PAC-learning axis-aligned rectangles is $O(\frac{1}{\epsilon} \log \frac{1}{\delta})$.

Generalization bound



- ▶ We can express the results above in terms of a generalization bound.
- ▶ A generalization bound states that with probability at least $1 - \delta$ $R(R_S)$ is upper bounded by some quantity that is a polynomial in the sample size m and δ .
- ▶ To get this generalization bound, we take the upper bound derived in the previous slides, set $\delta = 4e^{-\frac{m\epsilon}{4}}$ and solve for ϵ .
- ▶ Thus with probability at least $1 - \delta$ we find that
$$R(R_S) \leq \frac{4}{m} \log \frac{4}{\delta}.$$



- ▶ Our proof of PAC-learning applied to the tightest axis-aligned rectangle.
- ▶ Can we have a similar proof for axis-aligned rectangles that do not contain negative points? The answer is yes.
- ▶ What about other concept classes such as non-concentric circles instead of axis-aligned rectangles? It is hard to extend our proof to this case since our proof relies on a particular geometric argument.
- ▶ We provide a more general proof technique and more general results that apply to a finite hypothesis set. Note that the concept class of axis-aligned rectangles consists of an infinite number of hypotheses.

Guarantees for finite hypothesis sets - consistent case



- ▶ In the case of axis-aligned rectangles, the hypothesis h_S returned by the algorithm was always consistent, in that it correctly labeled all examples in S .
- ▶ We now present a general sample complexity bound that is intended for the case when the hypothesis space $\mathcal{H}$ is of finite size, and where each hypothesis is consistent.
- ▶ We will assume that the target concept $c \in \mathcal{H}$.

Learning bound - finite $\mathcal{H}$, consistent case



Theorem 2.5 Let $\mathcal{H}$ be a finite set of functions mapping from $\mathcal{X}$ to $\mathcal{Y}$. Let $\mathcal{A}$ be an algorithm that for any target concept $c \in \mathcal{H}$ and i.i.d sample S returns a consistent hypothesis h_S i.e $\hat{R}_S(h_S) = 0$. Then for any $\epsilon > 0, \delta > 0$ the inequality $\mathbb{P}_{S \in \mathcal{D}^m}[R(h_S) \leq \epsilon] \geq 1 - \delta$ holds if $m > \frac{1}{\epsilon}(\log |\mathcal{H}| + \log \frac{1}{\delta})$. This sample complexity result admits the following equivalent statement as a generalization bound: for any $\epsilon > 0, \delta > 0$, with probability at least $1 - \delta$, $R(h_S) \leq \frac{1}{m}(\log |H| + \log \frac{1}{\delta})$.

Learning bound - finite $\mathcal{H}$, consistent case



- ▶ Let us look at the proof of the theorem on the previous slide.
- ▶ Fix $\epsilon > 0$.
- ▶ We do not know which consistent hypothesis h_S is selected by the algorithm. The hypothesis depends on the training sample S .
- ▶ We therefore look for a uniform convergence bound that would hold for all consistent hypotheses.
- ▶ Thus we will bound the probability that some $h \in \mathcal{H}$ would be consistent and have an error greater than ϵ .

Learning bound - finite $\mathcal{H}$, consistent case



- ▶ Define $\mathcal{H}_\epsilon = \{h \in \mathcal{H} : R(h) > \epsilon\}$.
- ▶ The probability that a hypothesis $h \in \mathcal{H}_\epsilon$ is consistent on a training sample S containing m instances drawn i.i.d, can be bounded as follows: $\mathbb{P}[\hat{R}_S(h) = 0] \leq (1 - \epsilon)^m$. Since $R(h) > \epsilon$, the probability that the hypothesis h makes no error on any example from S is no more than $(1 - \epsilon)$. For the hypothesis to be consistent over S , ie be correct overall instances in S , the probability becomes bounded by $(1 - \epsilon)^m$ since all the instances are independent.



- By the union bound we have the following

$$\begin{aligned} & \mathbb{P}[\exists h \in \mathcal{H}_\epsilon, \hat{R}_S(h) = 0] \\ &= \mathbb{P}[\hat{R}_S(h_1) = 0 \vee \hat{R}_S(h_2) = 0 \dots \vee \hat{R}_S(h_{|\mathcal{H}_\epsilon|}) = 0] \\ &\leq \sum_{h \in \mathcal{H}_\epsilon} \mathbb{P}[\hat{R}_S(h) = 0] \\ &\leq \sum_{h \in \mathcal{H}_\epsilon} (1 - \epsilon)^m \leq |\mathcal{H}_\epsilon| (1 - \epsilon)^m \leq |\mathcal{H}_\epsilon| e^{-m\epsilon} \leq |\mathcal{H}| e^{-m\epsilon} \end{aligned}$$

- Setting the right hand side of the above inequality to be less than or equal to δ and solving for ϵ gives the result.

Learning bound - finite $\mathcal{H}$, consistent case



- ▶ The theorem shows that when the hypothesis set $\mathcal{H}$ is finite, a consistent algorithm $\mathcal{A}$ is a PAC-learning algorithm since the sample complexity is a polynomial in $\frac{1}{\delta}$ and $\frac{1}{\epsilon}$
- ▶ The generalization error of consistent hypothesis is bounded upwards by a term that decreases as the number of samples in S increases. This is as expected as we expect the learning algorithm to construct more accurate hypotheses as the number of labeled samples increases. However the generalization error being proportional to $\frac{1}{m}$ is a favourable result.

Learning bound - finite $\mathcal{H}$, consistent case



- ▶ The requirement imposed on algorithm $\mathcal{A}$ in that it must come up with a consistent hypothesis increases the size of the hypothesis set $|\mathcal{H}|$. This in turn increases the upper bound of the generalization error but the increase is only logarithmic in $|\mathcal{H}|$.
- ▶ We shall now use the above theorem to analyze the sample complexity of various concept classes.

Conjunction of Boolean literals



- ▶ Consider learning the concept class $\mathcal{C}_n$ consisting of conjunctions of at most n Boolean literals $x_1, x_2, \dots, x_n$.
- ▶ A Boolean literal is either a variable $x_i, i \in [n]$ or its complement $\bar{x}_i, i \in [n]$.
- ▶ For $n = 4$ an example is the conjunction $x_1 \wedge \bar{x}_2 \wedge x_4$.
(1, 0, 0, 1) is a positive example for this concept and
(1, 0, 0, 0) is a negative example.
- ▶ We can learn something about what literals are present in the concept that is a conjunction of literals by examining the literals found in a positive example.
- ▶ For example if (1, 0, 1, 0) is a positive example for the concept, the concept cannot contain the literals $\bar{x}_1$ and $\bar{x}_3$. Similarly it cannot contain the literals x_2 and x_4 either.

Conjunction of Boolean literals



- ▶ On the other hand a negative example is not so informative since it is not known which of the n bits are incorrect.
- ▶ We can come up with a simple consistent hypothesis based on positive examples.
- ▶ For each positive example $(b_1, b_2, \dots, b_n)$ if $b_i = 1$ then $\bar{x}_i$ is ruled out as a possible literal in the concept class. If $b_i = 0$ then x_i is ruled out.
- ▶ The conjunction of all the literals not ruled out is a hypothesis that is consistent with the target.
- ▶ What is the size of the concept class, i.e $|\mathcal{H}|$? We see that $|\mathcal{H}| = 3^n$ since for each literal there are three possibilities - include it in the concept positively, include it in the concept negatively or ignore it.

Conjunction of Boolean literals



- ▶ Plugging in the value of $|\mathcal{H}| = 3^n$ in the upper bound derived in Theorem 2.5 we have the following sample complexity bound for any $\epsilon > 0, \delta > 0$: $m \geq \frac{1}{\epsilon}((\log 3)n + \log \frac{1}{\delta})$.
- ▶ Thus the class of conjunctions of at most n Boolean literals is PAC-learnable.
- ▶ The computational complexity of $\mathcal{A}$ is also polynomial since the time spent per training example is $O(n)$.

Universal concept class



- ▶ Consider the set $\mathcal{X} = \{0, 1\}^n$ of all Boolean vectors with n components and let $\mathcal{U}_n$ be the concept class formed by all subsets of $\mathcal{X}$.
- ▶ Is this concept class PAC-learnable?
- ▶ To guarantee a consistent hypothesis the hypothesis class must include the concept class, thus $|\mathcal{H}| \geq \mathcal{U}_n = 2^{2^n}$.
- ▶ From Theorem 2.5 we obtain the following sample complexity bound: $m \geq \frac{1}{\epsilon} ((\log 2)2^n + \log \frac{1}{\delta})$.
- ▶ Thus the number of training samples required is exponential in n , which is the cost of representation of a single point in $\mathcal{X}$ - remember that $\mathcal{X}$ is the set of all n -tuples where each element is 0 or 1. This means that PAC-learning is not guaranteed by the theorem.



- ▶ A disjunctive normal form formula (DNF) is a disjunction of several terms, each of which is a conjunction of Boolean literals.
- ▶ A k -term DNF is a disjunction of k terms where each term is a conjunction of at most n literals.
- ▶ An example for $k = 2$ and $n = 3$ is $x_1x_3 + \bar{x}_1\bar{x}_2\bar{x}_3$.
- ▶ Let us examine the question of whether this concept class $\mathcal{C}$ of k -term DNF formulae is PAC learnable.
- ▶ The cardinality of the concept class of k term DNFs - the number of such DNFs - can be seen to be 3^{nk} as there are k terms in the disjunction and 3^n possible ways of constructing each conjunction of at most n literals.



- ▶ For consistency to be maintained, we need to have $|\mathcal{H}| \geq 3^{nk}$ since every possible hypothesis consisting of k terms should be included in the hypothesis set $\mathcal{H}$.
- ▶ Theorem 2.5 gives the following sample complexity bound:
 $m \geq \frac{1}{\epsilon}((\log 3)nk + \log \frac{1}{\delta})$ which is polynomial.
- ▶ The above appears to show that the concept class of k -term DNFs is PAC-learnable.
- ▶ However it is not **efficiently** PAC-learnable, unless the complexity class RP which represents those problems that have a randomized polynomial-time decision solution coincides with the class NP. This is widely thought to not be the case.

Guarantees for finite hypothesis sets - inconsistent case



- ▶ In the most general case, there may be no hypothesis in $\mathcal{H}$ that is consistent with the given sample S . What should we do in this case?
- ▶ This case may be the one that is most often seen in practice as the concept class may be more complex than the hypothesis set used for learning.
- ▶ Inconsistent hypotheses with a few errors on the training set can be useful and we can sometimes derive favourable guarantees under some assumptions.
- ▶ We will use Hoeffding's inequality in the form of the corollary stated on the next slide that relates the empirical error with the generalization error.

Guarantees for finite hypothesis sets - inconsistent case



Corollary 2.10 Fix $\epsilon > 0$. Then, for any hypothesis $h : X \rightarrow \{0, 1\}$, the following inequalities hold:

$$\mathbb{P}_{S \in \mathcal{D}^m}[\hat{R}_S(h) - R(h) \geq \epsilon] \leq e^{-2m\epsilon^2}$$

$$\mathbb{P}_{S \in \mathcal{D}^m}[\hat{R}_S(h) - R(h) \leq -\epsilon] \leq e^{-2m\epsilon^2}$$

By the union bound, the above implies the following two-sided inequality:

$$\mathbb{P}_{S \in \mathcal{D}^m}[|\hat{R}_S(h) - R(h)| \geq \epsilon] \leq 2e^{-2m\epsilon^2}$$

Guarantees for finite hypothesis sets - inconsistent case



Setting the right hand side of the two-sided inequality in the previous slide to δ and solving for ϵ gives us the following corollary:

Corollary 2.11 Fix a hypothesis $h : X \rightarrow \{0, 1\}$. Then for any $\delta > 0$, the following inequality holds with probability at least $1 - \delta$

$$R(h) \leq \hat{R}_S(h) + \sqrt{\frac{\log \frac{2}{\delta}}{2m}}$$

Example - tossing a coin



- ▶ Imagine tossing a biased coin that lands on its heads with probability p . Let our hypothesis be that the coins always leads on tails.
- ▶ Then the true error rate is $R(h) = p$ whereas our empirical error rate $\hat{R}_S(h) = \hat{p}$ where $\hat{p}$ is the empirical probability of heads based on the number of heads in the sample S consisting of independent, identically drawn samples.
- ▶ From Corollary 2.11, we are guaranteed that $|p - \hat{p}| \leq \sqrt{\frac{\log \frac{2}{\delta}}{2m}}$.
- ▶ If we choose $\delta = 0.02$ and use a sample size of 500, then with a probability $p \geq 0.98$, the following approximation quality is guaranteed for $\hat{p}$: $|p - \hat{p}| \leq \sqrt{\frac{\log 10}{1000}} \approx 0.048$.

Uniform convergence bound



- ▶ Can we apply Corollary 2.11 to bound the generalization error of a hypothesis h_S while training on a sample S ?
- ▶ No we cannot do so, because h_S would in this case not be a fixed hypothesis, but a random variable that varies with the training sample.
- ▶ For a fixed hypothesis, the expectation of its empirical error (i.e over different samples S) is the same as its generalization error.
- ▶ For a variable hypothesis h_S , the generalization error $R(h_S)$ is a random variable.
- ▶ Thus, as in the proof for the consistent hypothesis case, we need to derive a uniform convergence bound, that is a bound that holds with high probability over all hypotheses $h \in \mathcal{H}$.

Uniform convergence bound



Theorem 2.13 Let $\mathcal{H}$ be a finite hypothesis set. Then for any $\delta > 0$, with probability at least $1 - \delta$, the following inequality holds:

$$\forall h \in \mathcal{H}, R(h) \leq \hat{R}_S(h) + \sqrt{\frac{\log |\mathcal{H}| + \log \frac{2}{\delta}}{2m}}$$

Proof of Theorem 2.13



Let $h_1, h_2, \dots, h_{|\mathcal{H}|}$ be the hypotheses in the set $\mathcal{H}$. We can calculate $\mathbb{P}[\exists h \in \mathcal{H}, |R(h) - \hat{R}_S(h)| > \epsilon]$ as follows using the union bound and Corollary 2.11.

$$\begin{aligned} & \mathbb{P}[\exists h \in \mathcal{H}, |\hat{R}_S(h) - R(h)| > \epsilon] \\ &= \mathbb{P}[(|\hat{R}_S(h_1) - R(h_1)| > \epsilon) \vee (|\hat{R}_S(h_2) - R(h_2)| > \epsilon) \\ & \quad \vee \dots \vee (|\hat{R}_S(h_{|\mathcal{H}|}) - R(h_{|\mathcal{H}|})| > \epsilon)] \\ & \leq \sum_{h \in \mathcal{H}} \mathbb{P}[|\hat{R}_S(h) - R(h)| > \epsilon] \leq 2|\mathcal{H}|e^{-2m\epsilon^2} \end{aligned}$$

Setting the right hand side of the last inequality to be less than or equal to δ completes the proof.



- ▶ From the result of Theorem 2.13 we see that for a finite hypothesis set $\mathcal{H}$, $R(h) \leq \hat{R}_S(h) + O(\sqrt{\frac{\log_2 |\mathcal{H}|}{m}})$.
- ▶ The term $\log_2 |\mathcal{H}|$ can be interpreted as the number of bits needed to represent $\mathcal{H}$.
- ▶ A larger sample size m guarantees better generalization, and the bound increases with $|\mathcal{H}|$ but logarithmically.
- ▶ The bound here is a less favourable function of $\frac{\log_2 |\mathcal{H}|}{m}$ compared to the consistent case - here we use square-root which means that increasing the sample size is not as effective as in the consistent case.



- ▶ The bound suggests seeking a trade-off between reducing the empirical error versus controlling the size of the hypothesis set.
- ▶ A larger hypothesis set will penalize the second term but possibly reduce the first term.
- ▶ But for two hypotheses sets yielding the same empirical error it suggests using the smaller hypothesis set.
- ▶ This can be viewed as an application of Occam's razor principle suggested by William Occam - "plurality should not be posited without necessity" or that the simplest explanation is the best.

Decision trees and perceptrons



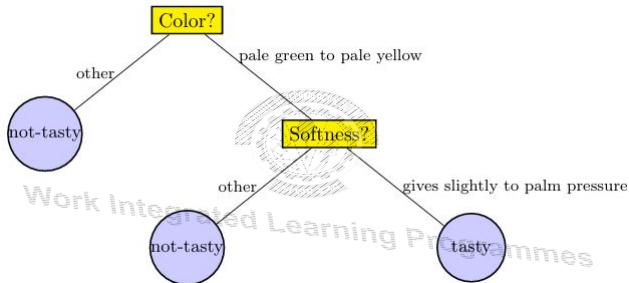
- ▶ We will study decision trees and perceptrons in this lecture.
- ▶ We will look at the minimum description length principle and how it is useful in looking at the difference between empirical error of a hypothesis and its risk.
- ▶ In the second half of this lecture we shall look at perceptrons.
- ▶ Some of the material in this lecture is drawn from the book Understanding Machine Learning: from Theory to Algorithms by S.S-Schwartz and Shai Ben-David.

Decision tree



- ▶ A decision tree is a predictor $h : \mathcal{X} \rightarrow \mathcal{Y}$ that predicts the label associated with an instance x by traveling through a tree from its root node to a leaf.
- ▶ We focus on the binary classification setting $\mathcal{Y} = \{0, 1\}$ but other settings are also possible.
- ▶ At each node on the path from root to a leaf, the input space is split based on one of the features of the instance x .
- ▶ A leaf contains a specific label which is then the label applied to the instance x .

Decision tree example



Decision tree example



- ▶ To test if a given papaya is tasty or not, the decision tree first checks the colour of the papaya. If the colour is not in the range pale green to pale yellow, the decision tree immediately predicts that the given papaya is not tasty without needing any additional tests.
- ▶ On the other hand, if the colour of the given papaya is in the range pale green to pale yellow, the decision tree conducts an additional check on the softness of the papaya. If it gives slightly to palm pressure the decision tree predicts that the papaya is tasty, otherwise not.
- ▶ We shall now study the sample complexity for a class of decision trees.



- ▶ A popular splitting rule at internal nodes of the tree is based on thresholding the value of a single feature.
- ▶ In other words, we move to the right child of a node or its left child based on the value of the indicator function $1_{x_i < \theta}$ where $i \in [d]$ is the index of the relevant feature and $\theta \in \mathbb{R}$ is the threshold.
- ▶ We can think of the decision tree as one that splits the input space into cells such that each leaf of the tree corresponds to one cell.
- ▶ For simplicity let us assume that $\mathcal{X} = \{0, 1\}^d$ so that each instance is a vector of d bits.

- ▶ In case we use $\mathcal{X} = \{0, 1\}$, it corresponds to thresholding according to a splitting rule of the form $1_{x_i=1}$ for some $i \in [d]$.
- ▶ With this simplifying assumption, let us calculate the size of the hypothesis class - ie the number of possible decision trees.
- ▶ Any classifier from $\{0, 1\}^d \rightarrow \{0, 1\}$ can be represented by a decision tree with 2^d leaves and a depth of $d + 1$ as we can label the leaves according to the classifier's label for the input instance that corresponds to the path from root to that particular leaf.
- ▶ This hypothesis set is very large, and its use might lead to overfitting the data. Can we reduce the size of the hypothesis set to something more reasonable?



- ▶ We use the principle that shorter hypotheses are better than longer hypotheses.
- ▶ To describe the kind of hypotheses that are of interest to us, we develop a description language.
- ▶ The description language is prefix-free in that no string in the language is a prefix for any other string in the language.
- ▶ The length of the description will mirror the size of the hypothesis - smaller strings will represent shorter hypotheses.

Prefix-free description



- ▶ We represent a tree of n nodes using a string consisting of $n + 1$ blocks, each of which contains $\log_2(d + 3)$ bits.
- ▶ The first n blocks of the tree encode the nodes of the tree listed in preorder and the last block indicates the end of the code.
- ▶ The label assigned to each block is a value that lies between 1 and $d + 3$.
- ▶ Any value between 1 and d denotes an internal node with the label indicating the variable on which splitting is performed, i.e if the label is k then splitting is performed on $1_{x_k=1}$.
- ▶ If the value is $d + 1$ it indicates that the node is a leaf node with value 1, $d + 2$ indicates it is a leaf node with value 0, and $d + 3$ indicates the END node.

Prefix-free description



- ▶ Since there are $d + 3$ labeling options for each node we will need $\log_2(d + 3)$ bits to encode the label for each node, and $(n + 1) \log_2(d + 3)$ bits to encode the description for the entire tree.
- ▶ The number of hypotheses that can be encoded is therefore $(d + 3)^{(n+1)}$.



- ▶ Following a result from the last lecture, specifically Theorem 2.13 we can relate the empirical error and the risk using the formula

$$\forall h \in \mathcal{H}, R(h) \leq \hat{R}_S(h) + \sqrt{\frac{\log |\mathcal{H}| + \log \frac{2}{\delta}}{2m}}$$

- ▶ Substituting for $|\mathcal{H}| = (d+3)^{n+1}$, we get

$$\forall h \in \mathcal{H}, R(h) \leq \hat{R}_S(h) + \sqrt{\frac{(n+1) \log_2(d+3) + \log \frac{2}{\delta}}{2m}}$$

Linear classification



- ▶ We now turn our attention to an algorithm for linear classification called the perceptron algorithm and analyze its generalization error.
- ▶ The problem is to separate a bunch of positive and negative points by a linear separator.
- ▶ We shall assume that the given problem is linearly separable, i.e there exists a linear separator which separates all positively labeled points from negative points.

Perceptron algorithm



PERCEPTRON($\mathbf{w}_0$)

```
1   $\mathbf{w}_1 \leftarrow \mathbf{w}_0$     ▷ typically  $\mathbf{w}_0 = \mathbf{0}$ 
2  for  $t \leftarrow 1$  to  $T$  do
3      RECEIVE( $\mathbf{x}_t$ )
4       $\hat{y}_t \leftarrow \text{sgn}(\mathbf{w}_t \cdot \mathbf{x}_t)$ 
5      RECEIVE( $y_t$ )
6      if ( $\hat{y}_t \neq y_t$ ) then
7           $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + y_t \mathbf{x}_t$  ▷ more generally  $\eta y_t \mathbf{x}_t, \eta > 0$ .
8      else  $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t$ 
9  return  $\mathbf{w}_{T+1}$ 
```

Perceptron algorithm



- ▶ The algorithm maintains a weight vector $w_t \in R^n$ that represents the hyperplane learned at time t . It starts with an arbitrary vector w_0 at time $t = 0$.
- ▶ At each round $t \in [T]$ it receives a point $x_t \in R^n$ and predicts its label using the current weight vector w_t .
- ▶ When the prediction does not match the actual label y_t of the point x_t it updates the weight vector w_t by adding $y_t x_t$ to it.
- ▶ When a learning rate $\eta > 0$ is used the weight vector w_t is updated by adding $\eta y_t x_t$ to it.



- ▶ The reason for the update can be understood as follows: when the algorithm produces a wrong label for the point x_t , $y_t w_t \cdot x_t$ is negative, so the weight vector w_t is updated to w_{t+1} such that $y_t w_{t+1} \cdot x_t = y_t (w_t + \eta y_t x_t) \cdot x_t = y_t w_t \cdot x_t + \eta y_t^2 \|x_t\|^2 = y_t w_t \cdot x_t + \eta \|x_t\|^2$. The idea therefore is to change the weight vector in such a way as to correct its mistake of misclassifying x_t .
- ▶ We can analyze this simple algorithm to provide an upper bound on the number of updates that will be made to the weight vector when a set of T points is presented to it.
- ▶ We will then use this result to obtain a result on the generalization error of the hypothesis learned.

Perceptron algorithm



Theorem 8.8: Let $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T \in \mathbb{R}^N$ be a sequence of T points with $\|\mathbf{x}_t\| \leq r$ for all $t \in [T]$, for some $r > 0$. Assume that there exist $\rho > 0$ and $\mathbf{v} \in \mathbb{R}^N$ such that for all $t \in [T]$ $\rho \leq \frac{\mathbf{y}_t \mathbf{v} \cdot \mathbf{x}_t}{\|\mathbf{v}\|}$. Then the number of updates made by the Perceptron algorithm when processing $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ is bounded by $\frac{r^2}{\rho^2}$.

Proof of Theorem 8.8



- ▶ Let J be the subset of the T rounds at which there is an update.
- ▶ Let M be the total number of updates so that $|J| = M$.
- ▶ Since $\rho \leq \frac{y_t \mathbf{v} \cdot \mathbf{x}_t}{\|\mathbf{v}\|}$, adding up the left and right hand side of these inequalities M times, once for each update, we have
$$M\rho \leq \frac{\mathbf{v} \cdot \sum_{t \in J} y_t \mathbf{x}_t}{\|\mathbf{v}\|}.$$
- ▶ We know that $\mathbf{v} \cdot \sum_{t \in J} y_t \mathbf{x}_t = \|\mathbf{v}\| \left\| \sum_{t \in J} y_t \mathbf{x}_t \right\| \cos \theta$ where θ is the angle between the vectors $\mathbf{v}$ and $\sum_{t \in J} y_t \mathbf{x}_t$. Since $\cos \theta \leq 1$ we have $\mathbf{v} \cdot \sum_{t \in J} y_t \mathbf{x}_t \leq \|\mathbf{v}\| \left\| \sum_{t \in J} y_t \mathbf{x}_t \right\|.$
- ▶ Thus we have $M\rho \leq \left\| \sum_{t \in J} y_t \mathbf{x}_t \right\|.$

Proof of Theorem 8.8



- ▶ Using the definition of updates we have $\mathbf{w}_{t+1} - \mathbf{w}_t = y_t \mathbf{x}_t$, so we can write $M\rho \leq \|\sum_{t \in J} y_t \mathbf{x}_t\| \leq \|\sum_{t \in J} \mathbf{w}_{t+1} - \mathbf{w}_t\|$.
- ▶ Since the last sum in the previous bullet point is a telescoping sum with $\mathbf{w}_0 = 0$ we get $\|\sum_{t \in J} \mathbf{w}_{t+1} - \mathbf{w}_t\| = \|\mathbf{w}_{T+1}\|$.
- ▶ Now $\|\mathbf{w}_{T+1}\| = \sqrt{\sum_{t \in J} \|\mathbf{w}_{t+1}\|^2 - \|\mathbf{w}_t\|^2}$.
- ▶ Now we can write $\|\mathbf{w}_{T+1}\| = \sqrt{\sum_{t \in J} \|\mathbf{w}_t + y_t \mathbf{x}_t\|^2 - \|\mathbf{w}_t\|^2}$.

Proof of Theorem 8.8



- ▶ Since $\|\mathbf{w}_t + y_t \mathbf{x}_t\|^2 - \|\mathbf{w}_t\|^2 = 2y_t \mathbf{w}_t \cdot \mathbf{x}_t + \|\mathbf{x}_t\|^2$ we can write
$$\|\mathbf{w}_{T+1}\| = \sqrt{\sum_{t \in J} \|\mathbf{w}_t + y_t \mathbf{x}_t\|^2 - \|\mathbf{w}_t\|^2} = \sqrt{\sum_{t \in J} 2y_t \mathbf{w}_t \cdot \mathbf{x}_t + \|\mathbf{x}_t\|^2}$$
- ▶ Since $2y_t \mathbf{w}_t \cdot \mathbf{x}_t \leq 0$, the last expression becomes
$$\sqrt{\sum_{t \in J} 2y_t \mathbf{w}_t \cdot \mathbf{x}_t + \|\mathbf{x}_t\|^2} \leq \sqrt{\sum_{t \in J} \|\mathbf{x}_t\|^2}.$$
- ▶ But we have an upperbound on the norm of any $\|\mathbf{x}_t\|$, i.e $\|\mathbf{x}_t\| \leq r$ so that $\sqrt{\sum_{t \in J} \|\mathbf{x}_t\|^2} \leq Mr^2$.
- ▶ Thus $M\rho \leq \sqrt{Mr^2}$ which yields $M \leq \frac{r^2}{\rho^2}$.

Implications of the result



- ▶ The weight vector after processing T points is a linear combination of the vectors $\mathbf{x}_t$ at which updates were made, i.e. $\mathbf{w}_T = \sum_{t \in J} y_t \mathbf{x}_t$.
- ▶ Using language from Support Vector Machines, these vectors can be referred to as support vectors.
- ▶ The bound on the number of updates established by Theorem 8.8 depends only on the normalized margin $\frac{\rho}{r}$ and not on the dimension N of the space.

Implications of the result



- ▶ The theorem makes no assumption on the sequence of points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$. A standard setting of the Perceptron algorithm is to choose some $m < T$ and let the algorithm make multiple passes over these m points.
- ▶ When the given set of points S is linearly separable, the algorithm will converge after a finite number of updates. For a small margin ρ the algorithm can converge very slowly.

Definition 5.2: Let h_S denote the hypothesis returned by a learning algorithm A when trained on a fixed sample S . Then the leave-one-out error of A on sample S of size m is defined by

$$\hat{R}_{LOO}(A) = \frac{1}{m} \sum_{i=1}^{i=m} 1_{h_{S-x_i}}(x_i) \neq y_i$$

- ▶ Thus for each $i \in [m]$ A is trained on all the points in S except x_i and its error is then computed using x_i . The leave-one-out error is based on the average of these errors.



- ▶ We explore the connection between Leave-one-out error and generalization error via Lemma 5.3 from Foundations of Machine Learning.

Lemma 5.3 The average leave-one-out error for samples of size $m \geq 2$ is an unbiased estimate of the average generalization error for samples of size $m - 1$.

$$E_{S \sim D^m}[\hat{R}_{LOO}(A)] = E_{S' \sim D^{m-1}}[R(h_{S'})]$$

Proof of Lemma 5.3



By the linearity of expectation we can write

$$\begin{aligned} E_{S \sim D^m}[\hat{R}_{LOO}(A)] &= \frac{1}{m} \sum_{i=1}^{i=m} E_{S \sim D^m}[1_{h_{S-x_i}}(x_i) \neq y_i] \\ &= E_{S \sim D^m}[1_{h_{S-x_1}}(x_1) \neq y_1] \\ &= E_{S' \sim D^{m-1}, x_1 \sim D}[1_{h_{S'}}(x_1) \neq y_1] \\ &= E_{S' \sim D^{m-1}}[E_{x_1 \sim D}[1_{h_{S'}}(x_1) \neq y_1]] \\ &= E_{S' \sim D^{m-1}}[R(h_{S'})] \end{aligned}$$

Leave-one-out error and perceptron updates



- ▶ We shall now investigate the relationship between data points in a sample S which contribute to the leave-one-out error and updates made by the perceptron algorithm.
- ▶ We shall show that if a point x_i contributes to leave-one-out error, ie $h_{S-x_i}(x_i) \neq y_i$, then the perceptron algorithm will definitely make at least one update on this point x_i .
- ▶ This observation will then be used in Theorem 8.9 to relate generalization error of the perceptron hypothesis with the number of updates made while learning the hypothesis.

Leave-one-out error and perceptron updates



- ▶ Let us prove the claim in the previous slide about how a point causing a leave-one-out error must necessarily cause an update during the running of the perceptron algorithm.
- ▶ Let $\mathbf{x}_i$ be such a point.
- ▶ Consider two runs of the perceptron algorithm with two sequences of points U and V such that $U = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{i-1}, \mathbf{x}_i, \mathbf{x}_{i+1}, \dots, \mathbf{x}_m)$ and $V = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{i-1}, \mathbf{x}_{i+1}, \dots, \mathbf{x}_m)$. The only difference between the sequences is that U contains $\mathbf{x}_i$ and V does not contain $\mathbf{x}_i$.
- ▶ If we consider that $\mathbf{x}_i$ causes a leave-one-out error, the hypothesis h_V learned by the perceptron algorithm is such that $h_V(\mathbf{x}_i) \neq y_i$.

Leave-one-out error and perceptron updates



- ▶ Now let us assume that in the sequence U there is no update caused by the point x_i . The subsequences leading up to the point x_{i-1} in both U and V are the same, so the weight vector after processing these subsequences in both U and V will be the same. Let us label this weight vector as w_{i-1} , i.e. this is the weight vector in both runs after processing the point x_{i-1} .
- ▶ The next point seen in sequence U is x_i . Since there is assumed to be no update at x_i we have $w_i = w_{i-1}$. Then when the perceptron algorithm processes the point x_{i+1} in U and in parallel processes the point x_{i+1} in V , the starting weight vector would be w_i in U and w_{i-1} in V . But we know that $w_{i-1} = w_i$.

Leave-one-out error and perceptron updates



- ▶ The subsequences in U and V starting from $\mathbf{x}_{i+1}$ are the same which means that if the algorithm runs both these subsequences with the same starting weight vector, it should end up with the same weight vector.
- ▶ But we know that in one case (running on V) the final hypothesis makes an error on $\mathbf{x}_i$ while in the other case (running on U) no error is made.
- ▶ This leads to a contradiction which means that there must have been an update at $\mathbf{x}_i$.
- ▶ Note that in general both algorithms running on U and V run on the same data multiple times. We can show that there must be an update on $\mathbf{x}_i$ in at least one of the iterations.

Leave-one-out error and perceptron updates



- ▶ From the preceding discussion we see that the leave-one-out-error $R_{LOO}(A)$ measured on a sample S of size $m + 1$ is less than the number of updates performed by the perceptron algorithm processing the same sample, i.e $M(S)$ divided by $m + 1$. Thus $R_{LOO}(A) \leq \frac{M(S)}{m+1}$.
- ▶ We can now state Theorem 8.9 which is a result that relates the generalization error of the hypothesis h_S which is learned by the Perceptron algorithm on processing a sample S of size m , to the number of updates made by the same algorithm as it processes the sample S .

Leave-one-out error and perceptron updates



Theorem 8.9: Assume that the data is linearly separable. Let h_S be the hypothesis returned by the Perceptron algorithm after training over a sample S of size m drawn from distribution D . Then the expected error of h_S is bounded as follows:

$$E_{S \sim D^m}[R(h_S)] \leq E_{S \sim D^{m+1}} \left[\frac{\min(M(S), \frac{r^2(S)}{\rho^2(S)})}{m+1} \right]$$

Proof of Theorem 8.9



- ▶ From Lemma 5.3 we showed that the expected value of the generalization error of hypothesis h_S is equal to the expected value of the leave-one-out error.
- ▶ Let S be a linearly separable sample of size $m + 1$ drawn i.i.d according to D and let $\mathbf{x}$ be a point in S . We have already seen that if $h_{S-\mathbf{x}}$ misclassifies $\mathbf{x}$ then $\mathbf{x}$ is a support vector for the perceptron algorithm which means that an update will be performed on it.
- ▶ Thus the leave-one-out error is bounded by $\frac{M(S)}{m+1}$. But Theorem 8.8 gives an upper bound on $M(S)$, so the result follows.



- ▶ We shall now look at online learning algorithms, the learning framework for which is in stark contrast to that of PAC learning.
- ▶ In PAC learning there are separate training and testing phases, but in online learning the training and testing phases are mixed up.
- ▶ Another point to note is that PAC learning follows the assumption that the data points are distributed in the same way over time, and that the training and test data are drawn from the same distribution.



- ▶ In PAC learning the natural goal is to learn a hypothesis that makes low generalization error.
- ▶ As far as online learning is concerned, no distributional assumption is made and there is no notion of generalization. Instead the performance of online algorithms is measured using a mistake model and a notion of regret.
- ▶ To derive guarantees in this model, we make some worst-case or adversarial assumptions.



- ▶ The general online learning setting involves T rounds. At the t th round, the algorithm receives an input $x_t \in \mathcal{X}$ and makes a prediction $\hat{y}_t \in \mathcal{Y}$.
- ▶ It then receives the true label for x_t , i.e y_t and therefore incurs a loss $L(\hat{y}_t, y_t)$ where $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$ is a loss function.
- ▶ It is possible in some cases for the prediction domain of the algorithm to be different from the true domain.
- ▶ For classification problems we have $\mathcal{Y} = \{0, 1\}$ and $L(y, y') = |y - y'|$. For regression problems $\mathcal{Y} \subset \mathbb{R}$ and typically we take $L(y, y') = (y - y')^2$.
- ▶ The objective in the online setting is to minimize the loss over T rounds, where the cumulative loss is $\sum_{i=1}^T L(y_i, y'_i)$.

Prediction with expert advice



- ▶ We shall discuss the setting of online learning with expert advice, and the associated notion of regret.
- ▶ In this setting the algorithm receives at the t th round not only the input instance $x_t \in \mathcal{X}$, but also advice from N experts, denoted $y_{t,i} \in \mathcal{Y}, i \in N$. The algorithm then makes a prediction, receives a true label and incurs a loss.
- ▶ After T rounds, the algorithm incurs a cumulative loss.

Prediction with expert advice



- ▶ The objective in this setting is to minimize the regret R_T , also called external regret, which compares the cumulative loss of the algorithm to that of the best expert in hindsight after T rounds. This regret is computed as follows:

$$R_T = \sum_{t=1}^{t=T} L(\hat{y}_t, y_t) - \min_{i=1}^{i=N} \sum_{t=1}^{t=T} L(\hat{y}_{t,i}, y_t)$$

Mistake bounds and Halving algorithm



- ▶ Let us assume that the loss function is the standard zero-one loss used in classification.
- ▶ We shall also make an assumption about the expert advice setting, namely that at least one of the experts makes no errors. This is called the realizable case.
- ▶ Under these circumstances, we can discuss the mistake bound model - namely, how many mistakes does the algorithm make before it learns the concept?
- ▶ Since we are in the realizable case, there will be some number T of rounds, after which we can be sure that the algorithm has learned the concept. Thereafter it will make no more errors.

Mistake bounds and Halving algorithm



- ▶ For any fixed concept c we can define the maximum number of mistakes a learning algorithm A makes as

$$M_A(c) = \max_{x_1, \dots, x_T} |\text{mistakes}(A, c)|$$

- ▶ For any concept c in a concept class $\mathcal{C}$, the maximum number of mistakes a learning algorithm A makes is

$$M_A(\mathcal{C}) = \max_{c \in \mathcal{C}} M_A(c)$$

- ▶ We would like to derive mistake bounds, that is, a bound on $M_A(\mathcal{C})$ starting with the Halving algorithm.

Halving algorithm



HALVING($\mathcal{H}$)

```
1   $\mathcal{H}_1 \leftarrow \mathcal{H}$ 
2  for  $t \leftarrow 1$  to  $T$  do
3      RECEIVE( $x_t$ )
4       $\hat{y}_t \leftarrow \text{MAJORITYVOTE}(\mathcal{H}_t, x_t)$ 
5      RECEIVE( $y_t$ )
6      if ( $\hat{y}_t \neq y_t$ ) then
7           $\mathcal{H}_{t+1} \leftarrow \{c \in \mathcal{H}_t; c(x_t) = y_t\}$ 
8      else  $\mathcal{H}_{t+1} \leftarrow \mathcal{H}_t$ 
9  return  $\mathcal{H}_{T+1}$ 
```

Halving algorithm



Theorem 8.1 Let $\mathcal{H}$ be a finite hypothesis set. Then
 $M_{\text{HALVING}}(\mathcal{H}) \leq \log_2 |\mathcal{H}|$



Proof of Theorem 8.1



- ▶ The proof given in the book assumes that each expert is a particular hypothesis, so that the number of experts and the number of hypotheses is the same.
- ▶ When the HALVING algorithm makes a mistake, it is because the majority of currently active hypotheses (or experts) got it wrong.
- ▶ These experts or hypotheses are then axed from the currently active set, so that the size of the currently active set is reduced by at least half each time a mistake is made.
- ▶ Thus the number of mistakes made can be no greater than $\log_2 |\mathcal{H}|$.

Proof of Theorem 8.1



- ▶ What happens if the number of experts $N > |\mathcal{H}|$ so that there could be a number of experts reflecting the same hypothesis?
- ▶ In this case, when a mistake happens we get rid of all of the experts that got it wrong, so that the remaining experts reflect hypotheses consistent with the actual observation.
- ▶ Let us assume that there are only 2 hypotheses in the set of all hypotheses and there are 1024 experts in total, one of whom represent the correct hypothesis and the other 1023 of whom represent the wrong hypothesis.

Proof of Theorem 8.1



- ▶ When the algorithm makes a mistake on a particular input we get rid of 1023 experts who got it wrong, and reduce the number of active hypotheses from 2 to 1.
- ▶ Thus the number of rounds is still no greater than $\log_2 |\mathcal{H}| = \log_2 2 = 1$ and not $\log_2 N$.
- ▶ Note that it is essential that the experts not be independent of the hypotheses - each expert must reflect in full the behaviour of some hypothesis.

Vapnik-Chervonenkis (VC)-dimension



- ▶ Can we develop a lower bound on the optimal number of mistakes in the mistake bound model?
- ▶ We can do so by considering the Vapnik-Chervonenkis dimension (VC-dimension) of the hypothesis set.
- ▶ To define the VC-dimension of a hypothesis space, we first introduce the concept of shattering.
- ▶ Given a hypothesis set $\mathcal{H}$, a dichotomy of a set S of points is one of the ways of labeling the points of S using a hypothesis in $\mathcal{H}$.
- ▶ A set S of $m \geq 1$ points is said to be shattered by a hypothesis set $\mathcal{H}$ when $\mathcal{H}$ realizes all possible dichotomies of the set S . That is when $\Pi_{\mathcal{H}}(m) = 2^m$.



- ▶ We will have occasion to look at the VC-dimension more closely later on in this course when we study Rademacher complexity.
- ▶ For now we shall use the VC-dimension to develop a lower bound for $opt(\mathcal{H})$, the optimal mistake bound for $\mathcal{H}$.
- ▶ We shall show that the VC-dimension of the hypothesis space $\mathcal{H}$ provides this lower bound.
- ▶ As an aside, we have already shown that $\log_2 |\mathcal{H}|$ provides an upper bound.

Lower bound on $opt(\mathcal{H})$



Theorem 8.2 Let $opt(\mathcal{H})$ be the optimal mistake bound for $\mathcal{H}$.
Then we have

$$VCdim(\mathcal{H}) \leq opt(\mathcal{H}) \leq M_{HALVING}(\mathcal{H}) \leq \log_2 |\mathcal{H}|.$$

Lower bound on $\text{opt}(\mathcal{H})$



- ▶ The fact that the VC-dimension of the hypothesis set $\mathcal{H}$ is d means that there is a set of d points for which the hypotheses in $\mathcal{H}$ realize all possible dichotomies.
- ▶ Let the learning algorithm A come up with a sequence s of 0 and 1 values as the labels for each of these d points.
- ▶ An adversary can choose the complementary sequence s' of 0s and 1s so as to contradict s for each point and cause d mistakes.

Lower bound on $opt(\mathcal{H})$



- ▶ Since the VC-dimension of $\mathcal{H}$ is d we can guarantee that s' is the output for some hypothesis in $\mathcal{H}$.
- ▶ $opt(\mathcal{H})$ is the smallest value of $M_A(\mathcal{H})$ over any learning algorithm A .
- ▶ $M_A(\mathcal{H}) = \max_{h \in \mathcal{H}} \max_{x_1, \dots, x_T} |mistakes(A, h)|$. Consider the set d of points which is shattered by the hypothesis set $\mathcal{H}$. A particular algorithm A would have a specific set of outputs on this set of d points and there is guaranteed to exist a true concept $h \in \mathcal{H}$ which is the direct opposite of this sequence. Therefore $M_A(\mathcal{H}) \geq d$. Another algorithm A' will show d mistakes for another true concept $h' \in \mathcal{H}$. Thus $M_A(\mathcal{H}) \geq d$ for all learning algorithms A . Since $opt(\mathcal{H}) = \min_A M_A(\mathcal{H})$ we have $opt(\mathcal{H}) \geq d$.

Weighted Majority algorithm



- ▶ By definition of optimality we know that $opt(\mathcal{H}) \leq M_{HALVING}(\mathcal{H})$ and we have already shown that $M_{HALVING}(\mathcal{H}) \leq \log_2 |\mathcal{H}|$.
- ▶ The HALVING algorithm simply discards experts after a single mistake on their part.
- ▶ We now move to the non-realizable setting and use a more general and less extreme algorithm called the WEIGHTED MAJORITY (WM) algorithm that assigns weights to the experts that are based on the mistakes made by them.
- ▶ The WM algorithm begins with uniform weights for all experts.
- ▶ At each round, it generates predictions based on a weighted majority of the opinions of all the experts.



- ▶ After receiving the true label, the algorithm reduces the importance of incorrect experts by multiplying their weights by a factor $\beta \in [0, 1)$, i.e $0 \leq \beta < 1$.
- ▶ This algorithm generalizes the HALVING algorithm since when $\beta = 0$ it reduces to the HALVING algorithm.
- ▶ Analysis of this algorithm cannot be done by extending the analysis of the HALVING algorithm since we work in the non-realizable setting.
- ▶ We present a theorem that presents a bound m_T on the number of mistakes made by the algorithm after $T \geq 1$ rounds of online learning as a function of the number of mistakes made by the best expert, or the expert that makes the fewest mistakes on the input sequence $y_1, y_2, \dots, y_T$.

WEIGHTED-MAJORITY(N)

```
1  for  $i \leftarrow 1$  to  $N$  do
2       $w_{1,i} \leftarrow 1$ 
3  for  $t \leftarrow 1$  to  $T$  do
4      RECEIVE( $x_t$ )
5      if  $\sum_{i: y_{t,i}=1} w_{t,i} \geq \sum_{i: y_{t,i}=0} w_{t,i}$  then
6           $\hat{y}_t \leftarrow 1$ 
7      else  $\hat{y}_t \leftarrow 0$ 
8      RECEIVE( $y_t$ )
9      if  $(\hat{y}_t \neq y_t)$  then
10         for  $i \leftarrow 1$  to  $N$  do
11             if  $(y_{t,i} \neq y_t)$  then
12                  $w_{t+1,i} \leftarrow \beta w_{t,i}$ 
13             else  $w_{t+1,i} \leftarrow w_{t,i}$ 
14  return  $w_{T+1}$ 
```


Theorem 8.3 Fix $\beta \in [0, 1]$. Let m_T be the number of mistakes made by algorithm WM after $T \geq 1$ rounds, and m_T^* be the number of mistakes made by the best of the N experts. Then, the following inequality holds:

$$m_T \leq \frac{\log N + m_T^* \log \frac{1}{\beta}}{\log \frac{2}{1+\beta}}$$

Proof of Theorem 8.3



- ▶ Let us assume that all experts start with a weight equal to 1. Let us denote $w_{t,i}$ denote the weight of the i th expert after t rounds.

- ▶ Let
$$W_t = \sum_{i=1}^{i=N} w_{t,i}.$$

- ▶ Assume that the algorithm makes an error at round t . This means that the weight of the incorrect experts at round t is greater than the weight of the correct experts. We can write the total weight of correct experts at round t as $\frac{W_t}{2} - r$ and the total weight of wrong experts as $\frac{W_t}{2} + r$, where $r > 0$ since we know the algorithm has made a mistake.

Proof of Theorem 8.3



- ▶ Now at round $t + 1$, the weights of all correct experts stays the same while those of incorrect experts get scaled by $\beta \in [0, 1]$.
- ▶ Thus we can write $W_{t+1} = (\frac{W_t}{2} - r) + \beta(\frac{W_t}{2} + r)$. This can be further simplified to $\frac{W_t}{2} + \beta \frac{W_t}{2} + (\beta - 1)r$. The last expression is less than $\frac{W_t}{2} + \beta \frac{W_t}{2}$ because $r \geq 0$ and $0 < \beta \leq 1$ which means $W_{t+1} \leq \frac{1+\beta}{2} W_t$.
- ▶ Since $W_1 = N$ we can write $W_T \leq \left(\frac{1+\beta}{2}\right)^{m_T} N$

Proof of Theorem 8.3



- ▶ Since all the weights are initially equal to 1, the weight of the i th expert after T rounds, it must be true that $w_{T,i} = \beta^{m_{T,i}}$ where $m_{T,i}$ is the number of mistakes made by the i th expert after T rounds.
- ▶ Moreover since W_T is the sum of all the weights of the experts after T rounds, we must have $W_T \geq w_{T,i}$.
- ▶ Let us assume that the number of mistakes made by the best expert is m_T^* .

- ▶ Then the following holds: $\beta^{m_T^*} \leq W_T \leq \left(\frac{1+\beta}{2}\right)^{m_T} N$.

Rearranging this equation gives

$$m_T \log \frac{2}{1+\beta} \leq \log N + m_T^* \log \frac{1}{\beta}.$$